

Strategy



Notas organizadas a partir dos slides e das anotações manuscritas de aula, Setembro/2026.

Disciplina: Métodos Avançados de Programação (MAP) — UEPB, 26.2

Professora: Luciana de Queiroz Leal Gomes

Referências: Larman (Utilizando UML e Padrões) · GoF (Padrões de Projeto) · Freeman & Freeman (Use a Cabeça! Padrões de Projetos) · Bloch (Java Efetivo)

Material Auxiliar

Padrões de Projeto GoF

- [Strategy no Refactoring Guru](#) (site indicado em aula para estudar os padrões)

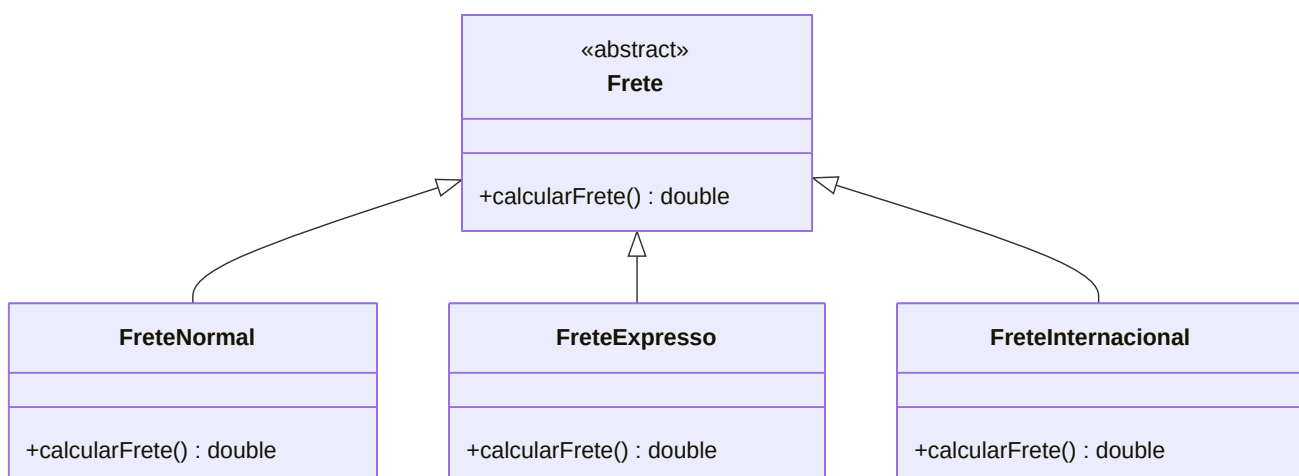
1. Herança × Composição ★

Antes de ver o padrão, as anotações retomam dois modos de reaproveitar e organizar comportamento em POO. O Strategy é, no fundo, uma escolha deliberada de um deles.

1.1 Herança (relação "é um")

- Em Java, usa-se `extends` (entre classes concretas e abstratas). A subclasse **herda** atributos e métodos da superclasse.
- Usa-se herança principalmente para **reaproveitar código** e para modelar uma relação do tipo "é um" (um `FreteExpresso` **é um** `Frete`).
- Ela viabiliza o **polimorfismo**: uma variável do tipo da superclasse pode guardar um objeto de qualquer subclasse.
- É chamada de reutilização **caixa-branca**: a subclasse enxerga (e depende de) detalhes internos da superclasse, como membros `protected`. Por isso **a herança prende as duas classes fortemente** (GAMMA et al., 2000).

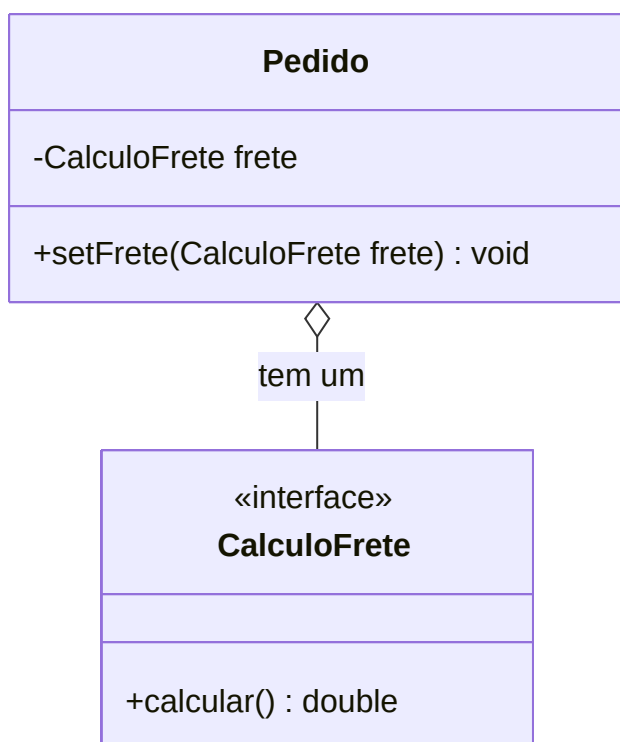
Diagrama simples de superclasse e subclasse



Leitura: a seta (triângulo vazado) aponta **da subclasse para a superclasse**. Frete é abstrata e declara `calcularFrete()` ; cada subclasse dá a sua própria versão.

1.2 Composição (relação "tem um")

- Uma classe contém uma referência a outro objeto e delega trabalho a ele. É a relação **"tem um"** (um `Pedido` **tem um** cálculo de frete).
- É reutilização **caixa-preta**: quem usa o objeto conhece **apenas a interface** (o contrato), **não os detalhes internos** (GAMMA et al., 2000).
- Em UML, aparece como **agregação** (losango vazado) ou **composição** (losango preenchido); a diferença está no ciclo de vida (ver Dúvida 3, seção 9).



Quem usa o objeto **não precisa** (e **não deve**) depender de como ele funciona por dentro; confia apenas no contrato da interface. É justamente isso que reduz o acoplamento.

1.3 Comparação

Característica	Herança	Composição
Relação	"É um"	"Tem um"
Palavra-chave em Java	<code>extends</code>	Atributo + referência a uma interface
Quando o vínculo é definido	Ao escrever o código (compilação)	Pode ser definido ou trocado na execução

Característica	Herança	Composição
Tipo de reuso	Caixa-branca (vê os detalhes internos)	Caixa-preta (só vê a interface)
Acoplamento	Forte entre superclasse e subclasse	Fraco (depende só de uma abstração)
Flexibilidade	Baixa: mudar o comportamento exige outra subclasse	Alta: basta trocar o objeto
Risco típico	Explosão de subclasses; mudança na base quebra as derivadas	Mais objetos para gerenciar

1.4 Princípios que sustentam o Strategy

Os livros da disciplina resumem a motivação em três diretrizes que aparecem juntas no Strategy:

- **Separar o que varia do que permanece igual**, isolando os aspectos que mudam (FREEMAN; FREEMAN, 2009). É o "*pensamento orientado à mudança*".
- **Programar para uma interface, não para uma implementação** (FREEMAN; FREEMAN, 2009; GAMMA et al., 2000).
- **Dar preferência à composição em vez da herança** (GAMMA et al., 2000; FREEMAN; FREEMAN, 2009; BLOCH, 2018, item 18).

Do lado do GRASP (Larman), o Strategy concretiza o padrão **Variações Protegidas** (proteger o sistema dos pontos que variam por meio de uma interface estável) e usa o padrão **Polimorfismo** (LARMAN, 2007). Isso liga esta aula ao "GRASP orienta decisões, GoF concretiza soluções" da Aula 1 da prof.

2. O problema: cálculo de frete em um e-commerce

2.1 O cenário

Imagine um sistema de e-commerce que precisa **calcular o frete**. Existem várias formas de cálculo:

- frete **normal**;
- frete **expresso**;
- frete **internacional**;
- frete **promocional**;

- frete **por região**.

o que acontece quando surgem novas regras?

2.2 Solução 1 (ingênua): if/else

```
if (tipo == NORMAL) {  
    // cálculo  
} else if (tipo == EXPRESSO) {  
    // cálculo  
} else if (tipo == INTERNACIONAL) {  
    // cálculo  
}
```

Problemas:

- **Muitos if/else.**
- **Difícil manutenção.**
- **Toda mudança quebra algo:** cada novo frete exige editar este bloco, e um deslize afeta os fretes que já funcionavam.
- **Código gigante.**
- **Alto acoplamento:** tudo fica dependente de tudo dentro do mesmo trecho.

Isso é o mau cheiro dos "muitos if s"

2.3 a god class

O sistema **mistura** três coisas na mesma classe:

- **regra de negócio;**
- **algoritmo;**
- **controle do sistema.**

O resultado é a **classe "faz tudo" (god class)**: uma classe enorme, que centraliza muitas responsabilidades e controla muitos objetos diferentes. **É um anti-padrão.**

Por que é tão ruim: uma god class tem **baixa coesão** (várias responsabilidades sem relação) e **alto acoplamento** (todo mundo depende dela), o oposto do mantra da POO.

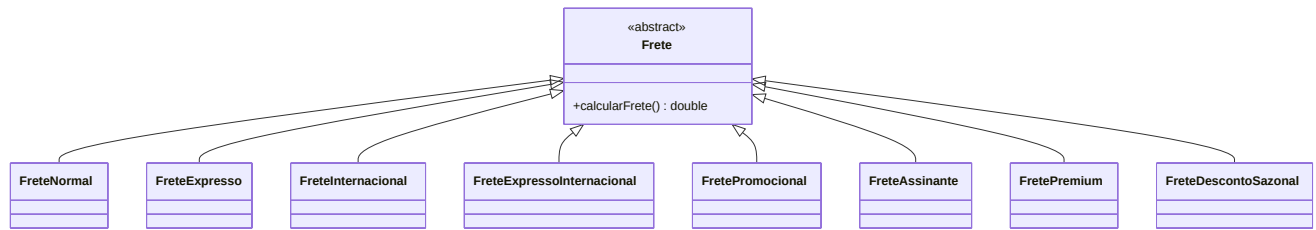
2.4 Solução 2: tentar resolver com herança

O desenvolvedor pensa: "vou criar subclasses!". Parte-se de uma superclasse `Frete` com `calcularFrete()` e uma subclasse por tipo (o diagrama da seção 1.1).

Explosão de subclasses

O problema aparece quando surgem **novas combinações**:

- frete expresso internacional;
- frete promocional;
- frete para assinantes;
- frete premium;
- frete com desconto sazonal.



O **número de subclasses explode**. A causa é que as variações são **independentes** (tipo de entrega, promoção, perfil do cliente) e a herança obriga a criar **uma classe para cada combinação**.

Ilustração numérica: com 3 tipos de entrega, 2 promoções e 2 perfis de cliente, a herança pede até $3 \times 2 \times 2 = 12$ subclasses; separando cada variação em uma peça própria, seriam $3 + 2 + 2 = 7$ peças combináveis.

O sistema fica rígido

Toda nova regra exige:

- **criar subclasses;**
- **alterar código existente;**
- **recompilar partes do sistema;**
- **risco de quebrar funcionalidades.**

A manutenção começa a ficar cara.

2.5 O verdadeiro problema ★

- O problema **NÃO** é o cálculo do frete.
- O problema é o **comportamento variável**: o algoritmo muda frequentemente.

Essa mudança de olhar é o que leva ao padrão: o que precisa ser isolado não é "o frete", e sim **a parte que varia**.

3. Pensando na solução

e se o comportamento pudesse ser...

- **separado da classe principal?**
- **trocado dinamicamente?**

- reutilizado?
- combinado?

A resposta é: em vez de colocar o algoritmo **dentro** da classe, **coloca-se o algoritmo em outro objeto**. Ou seja, **transforma-se comportamento em objeto**.

3.1 Consequências dessa ideia

Agora é possível:

- **trocar algoritmos em tempo de execução**;
- **reutilizar** comportamentos;
- **eliminar if/else** (e o código duplicado);
- **reduzir subclasses**;
- **desacoplar** regras.

Em **tempo de compilação**, a classe principal passa a depender **apenas de uma interface**, e não de cada algoritmo concreto (menos dependências de compilação). A **escolha do algoritmo** só acontece em **execução**.

4. O padrão Strategy

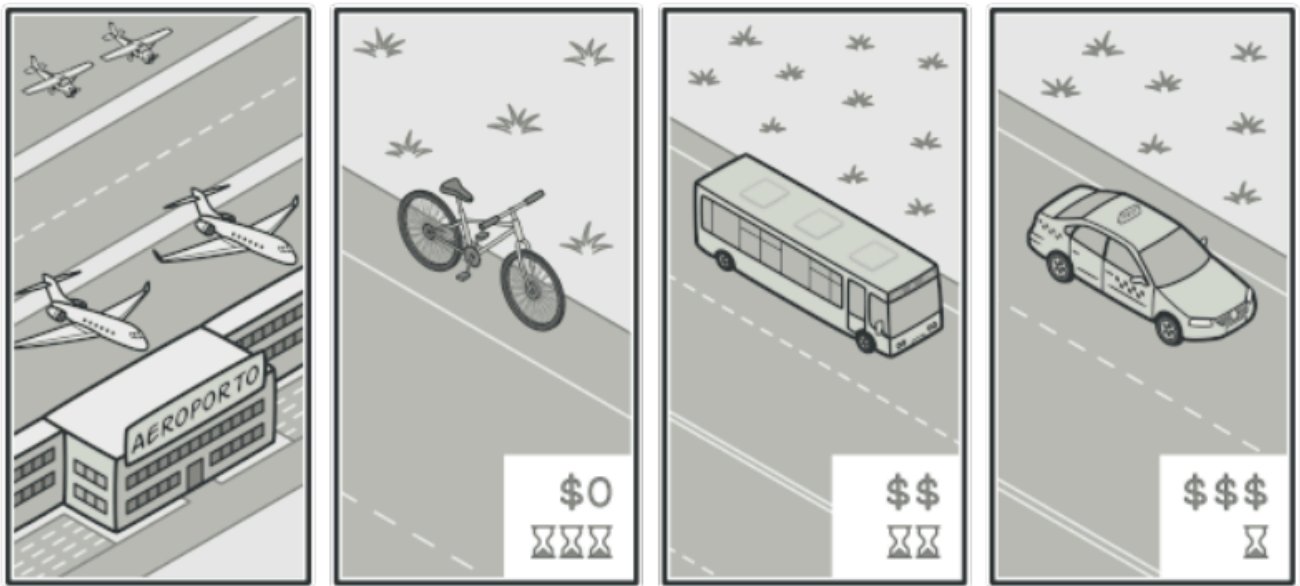
4.1 Definição oficial do GoF ★

O padrão Strategy "**define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis**."

O Strategy surge exatamente para resolver **variações de comportamento de forma flexível e elegante**. Segundo o GoF, ele permite que o algoritmo **varie independentemente dos clientes que o utilizam** (GAMMA et al., 2000).

4.2 Ideia central

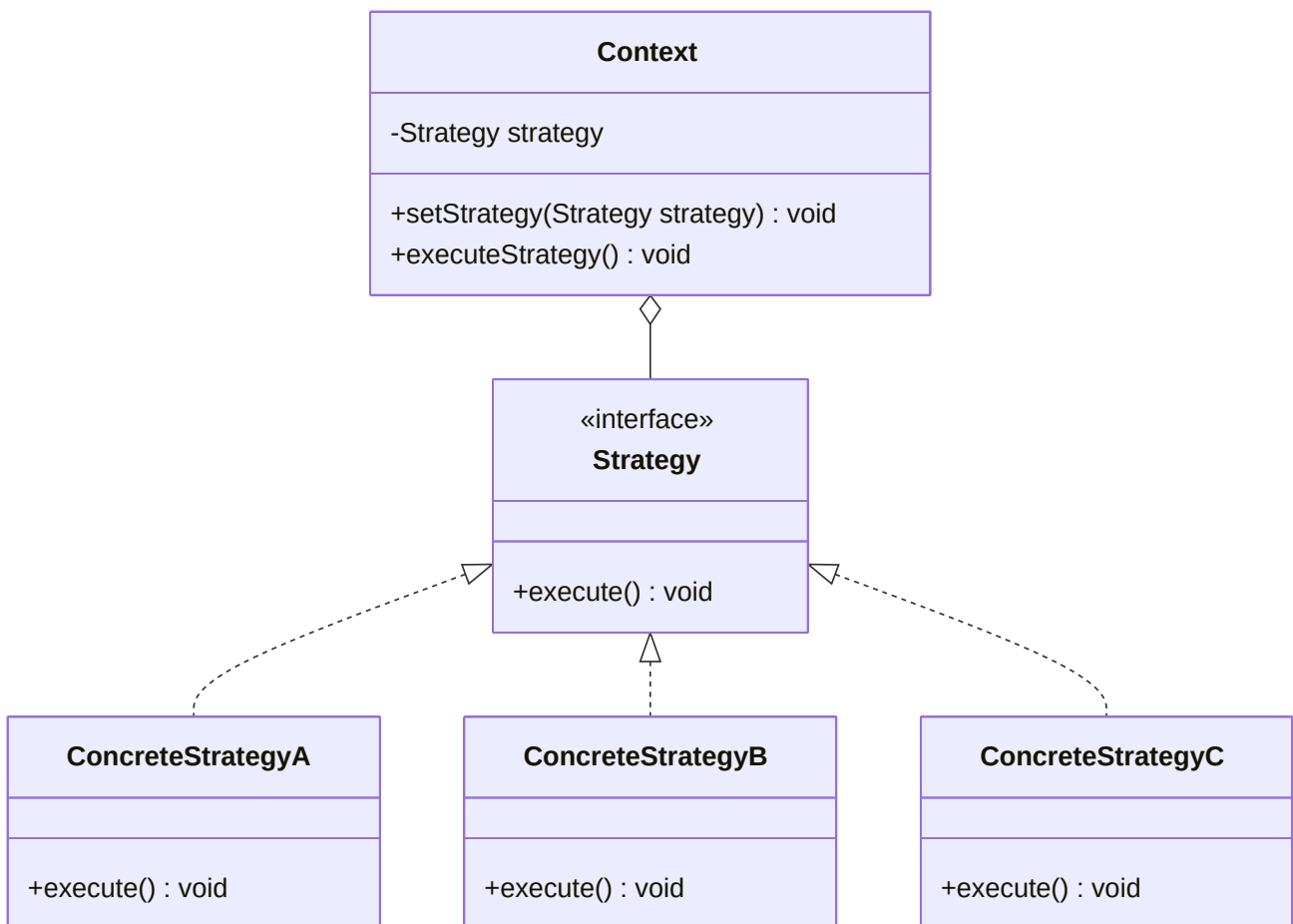
- O Strategy diz: "**coloque cada algoritmo em uma classe separada**".
- Cada comportamento vira **um objeto**.
- Os algoritmos ficam **independentes** entre si.
- É possível **trocar algoritmos em tempo de execução**.
- **Encapsula comportamento** e permite trocar algoritmos (estratégias) dinamicamente.

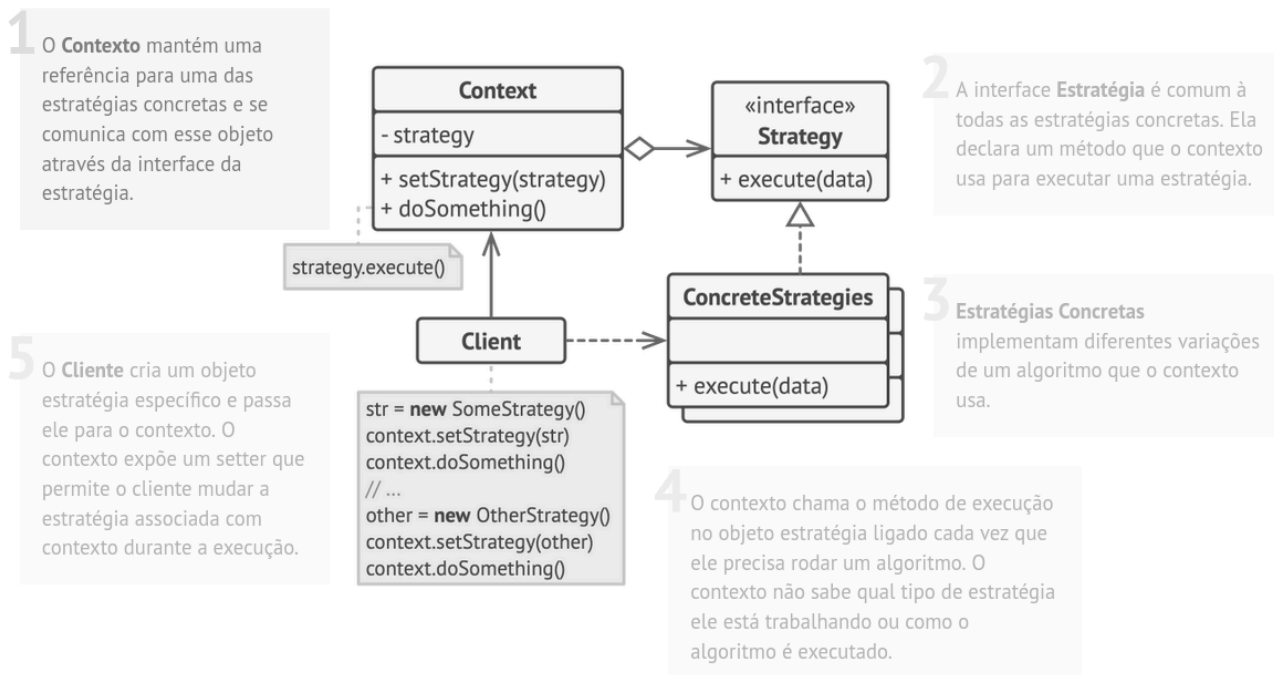


A ilustração mostra formas de chegar ao aeroporto (bicicleta, ônibus, táxi): o objetivo é o mesmo, cada estratégia tem custo e tempo diferentes e pode ser escolhida conforme o contexto.

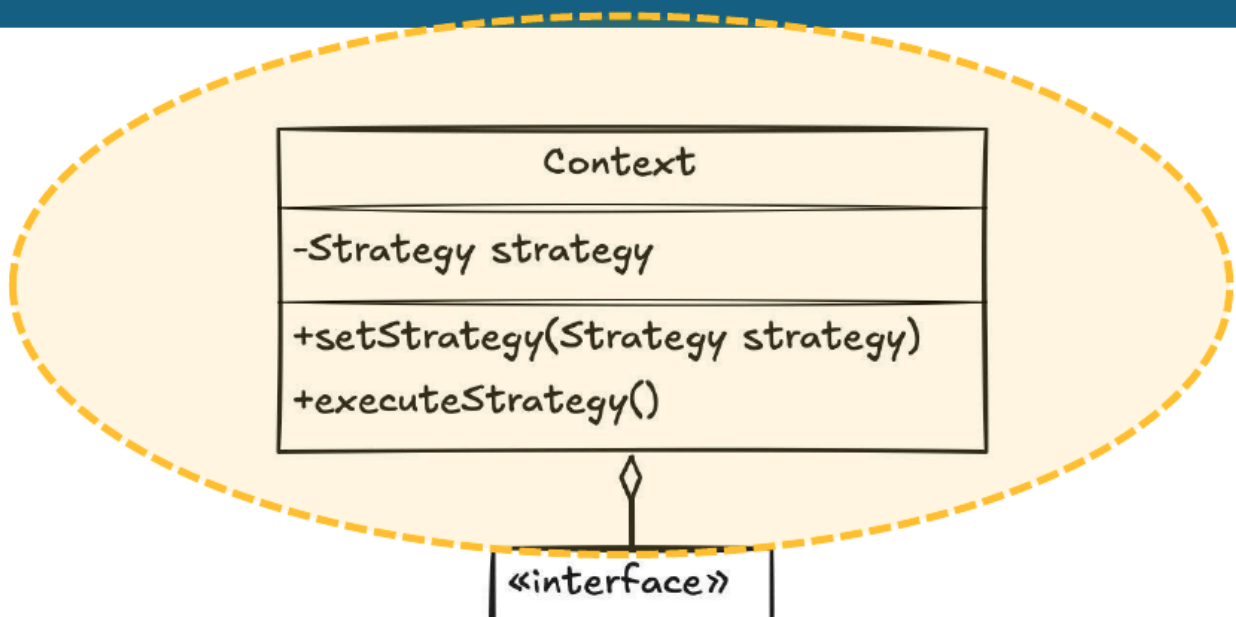
4.3 Participantes e estrutura (UML)

Todo padrão de projeto possui um diagrama UML que representa o seu funcionamento. O do Strategy é:





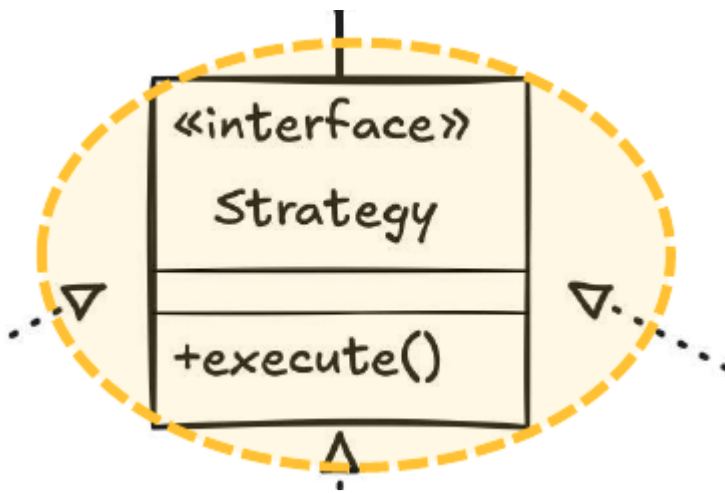
O Context



- **Usa** a estratégia.
- **NÃO sabe detalhes** de implementação.
- **Apenas delega** a execução. (Isso é MUITO importante)

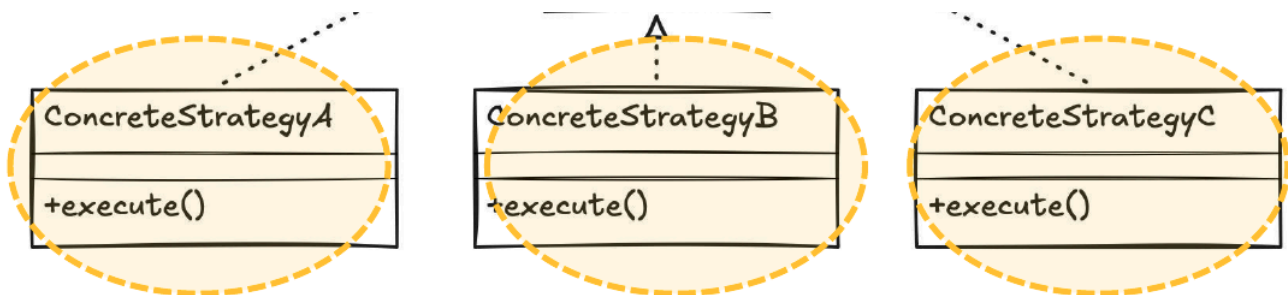
Por quê: se o `Context` conhecesse o algoritmo, voltaria o `if/else` e a necessidade de alterá-lo a cada regra nova.

A Strategy abstrata



- É a **interface comum** dos algoritmos.
- Define **O QUE** deve ser feito, mas **NÃO COMO**.
- A classe principal depende da **abstração** e não das implementações concretas, o que **reduz o acoplamento**.

As estratégias concretas



- São as **implementações reais** do algoritmo.
- Quem define o "**como**" são elas.
- ✎ Para criar um novo comportamento, basta **criar mais uma classe concreta que implementa a interface**, sem mexer no resto.

4.4 O segredo: composição, não herança ★



- O segredo do Strategy é **composição**, e não herança.
- O objeto **recebe** um comportamento (ele **não tem** um comportamento embutido).

Ou seja, o comportamento **não é herdado** de uma superclasse nem escrito dentro da classe: é **entregue de fora**, na forma de um objeto. Isso é o que permite trocá-lo (GAMMA et al., 2000; FREEMAN; FREEMAN, 2009).

4.5 Interface ou classe abstrata para a Strategy?

-  Poderia-se usar uma **classe abstrata** para representar a Strategy, mas com **interface** desacopla-se ainda mais.
- **Uso como tipo:** Lembrando que, pode-se declarar uma variável com o **tipo da interface** e atribuir a ela uma instância de qualquer classe que a implemente:

```
CalculoNota c = new MediaSimples(); // variável do tipo da interface
c = new MediaPonderada();           // outro objeto, mesmo tipo
```

- **Implementação por classes:** para usar uma interface, é preciso criar uma **classe concreta que a implemente** e forneça o código de **todos** os métodos definidos.
-  **Polimorfismo:** quando o Context chama calcular() **através da interface**, é executada a versão da estratégia concreta que estiver guardada naquele momento.

Lembrando que interface tipicamente contém métodos abstrados e constantes (não podem ter atributos). Em Java **desde a versão 8** uma interface também pode ter métodos default e static.

4.6 Troca dinâmica de comportamento (em tempo de execução)



- Uma das maiores forças do Strategy é **trocar algoritmos em tempo de execução**.
- O sistema pode **mudar de comportamento DURANTE a execução** do programa.
- **Sem alterar código, sem criar subclasses, sem if/else gigantes.**

O detalhamento de "o que é tempo de execução" e "por que isso importa" está na **Dúvida 1** (seção 9), que responde à pergunta das anotações.

5. Sistema de controle acadêmico

5.1 O cenário

Um sistema de controle acadêmico precisa calcular:

- aprovação **simples**;
- aprovação por **média ponderada**;
- aprovação **com prova final**.

Cada universidade usa uma regra. (Ou seja, a regra é variável e vem do contexto de uso.)

5.2 Sem Strategy

```
class Participacao {
    calcularNotaFinal() {
        if (tipo == MEDIA) { ... }
        else if (tipo == PONDERADA) { ... }
        else if (tipo == FINAL) { ... }
    }
}
```

Problema: toda nova regra altera a classe Participacao .

5.3 Com Strategy: cada cálculo vira uma estratégia

A interface (Strategy)

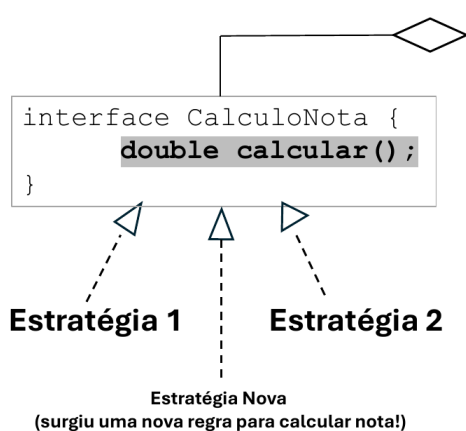
```
interface CalculoNota {
    double calcular();
}
```

As estratégias concretas

```
class MediaSimples implements CalculoNota {
    public double calcular() {
        return (n1 + n2) / 2;
    }
}

class MediaPonderada implements CalculoNota {
    public double calcular() {
        return n1 * 0.4 + n2 * 0.6;
    }
}
```

O Context



```
class Participacao {
    private CalculoNota estrategia;

    public Participacao(CalculoNota estrategia) {
        this.estrategia = estrategia;
    }

    public double resultado() {
        return estrategia.calcular(...);
    }
}
```

```

class Participacao {
    private CalculoNota estrategia;

    public Participacao(CalculoNota estrategia) {
        this.estrategia = estrategia;
    }

    public double resultado() {
        return estrategia.calcular(...);
    }
}

```

O atributo `estrategia` tem o **tipo da interface** (`CalculoNota`), e o `Context` (`Participacao`) só **delega** em `resultado()`. Quem decide qual algoritmo usar é quem cria a `Participacao` e escolhe a estratégia passada ao construtor.

5.4 Versão completa e executável

Uma versão que roda, passando as notas como parâmetro:

```

interface CalculoNota {
    double calcular(double n1, double n2);
}

class MediaSimples implements CalculoNota {
    public double calcular(double n1, double n2) { return (n1 + n2) / 2; }
}

class MediaPonderada implements CalculoNota {
    public double calcular(double n1, double n2) { return n1 * 0.4 + n2 * 0.6; }
}

class Participacao {
    private CalculoNota estrategia;
    private final double n1, n2;

    Participacao(CalculoNota estrategia, double n1, double n2) {
        this.estrategia = estrategia;
        this.n1 = n1;
        this.n2 = n2;
    }

    void setEstrategia(CalculoNota estrategia) { this.estrategia = estrategia; }

    double resultado() { return estrategia.calcular(n1, n2); }
}

```

```

public class Main {
    public static void main(String[] args) {
        Participacao p = new Participacao(new MediaSimples(), 6, 8);
        System.out.println(p.resultado());    // 7.0

        p.setEstrategia(new MediaPonderada());    // troca em tempo de
execução
        System.out.println(p.resultado());    // aproximadamente 7.2
    }
}

```

5.5 O que aconteceu aqui?

A classe principal:

- **NÃO** conhece o algoritmo;
- **NÃO** possui `if/else`;
- **NÃO** precisa mudar quando surge uma regra nova;
- apenas delega.

*O sistema fica **ABERTO para extensão e FECHADO para modificação**. Esse é o **Princípio Aberto-Fechado*** (FREEMAN; FREEMAN, 2009): estende-se o sistema criando uma nova estratégia, sem **editar** o código que já funciona.

5.6 Trocando o algoritmo em execução

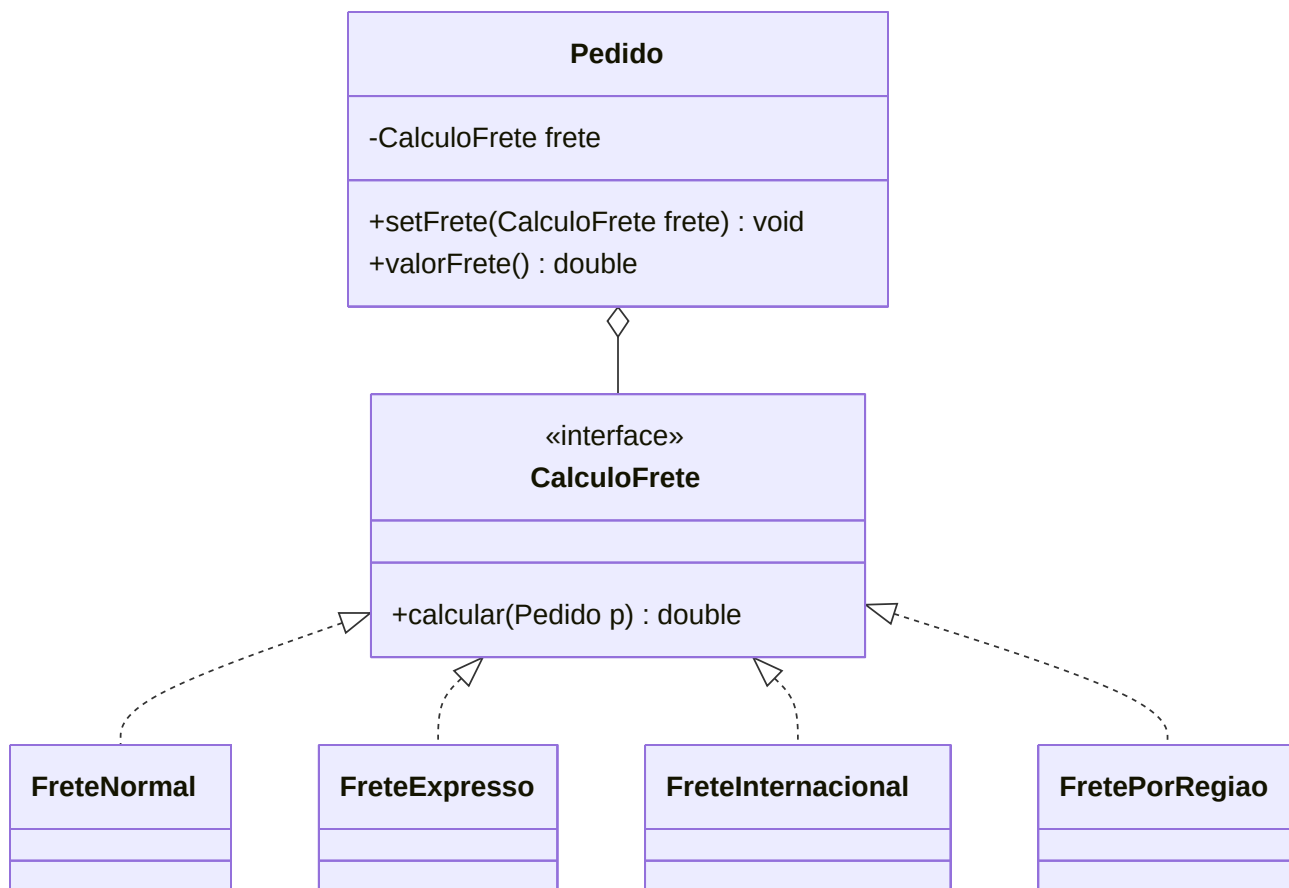
```

participacao.setEstrategia(new MediaPonderada());

```

5.7 Voltando ao problema do frete

O mesmo raciocínio resolve o problema:



- **Pedido** deixa de ter `if/else`: só delega em `frete.calcular(this)`.
- Um novo tipo de frete (por exemplo, "frete premium") vira **uma nova classe** que implementa **CalculoFrete**, sem tocar em **Pedido** nem nos outros fretes.
- Cada classe é **pequena e coesa**: tem **um único comportamento**.

Não teremos mais explosão de subclasses porque teremos apenas classes que têm um método. Mas lembrando que o Strategy **não elimina** a criação de várias classes. O que ele evita é a **explosão combinatória de subclasses** e o `if/else` gigante: as classes continuam existindo, mas são **pequenas, independentes e combináveis**.

6. Vantagens e desvantagens

6.1 Vantagens

Vantagem	Por quê
Remove <code>if/else</code> gigantes	A escolha do algoritmo passa a ser feita pela troca do objeto , não por condicionais (GAMMA et al., 2000).
Facilita manutenção	Cada algoritmo fica isolado; mexer em um não afeta os outros .

Vantagem	Por quê
Reduz acoplamento	O <code>Context</code> depende só da interface.
Facilita testes	Cada estratégia pode ser testada isoladamente, e o <code>Context</code> pode receber uma estratégia falsa.
Permite reutilização	A mesma estratégia serve a vários <code>Context</code> s.
Facilita expansão	Novas regras entram como novas classes (Aberto-Fechado).

6.2 Desvantagens

Desvantagem	Explicação
Muitas classes	Cada algoritmo é uma classe; o número de objetos cresce (GAMMA et al., 2000).
O cliente precisa conhecer as estratégias	Quem escolhe a estratégia (o código que monta o <code>Context</code>) precisa saber quais existem e qual usar (GAMMA et al., 2000).
Pode aumentar a complexidade	Se a variação é pequena, o padrão pode ser mais estrutura do que o problema pede.

7. Quando usar e quando não usar ★

7.1 Use o Strategy quando

- existem muitos algoritmos parecidos;
- o comportamento varia;
- há muitos `if/else`;
- novas regras surgem frequentemente.

7.2 NÃO use o Strategy quando

- existe apenas um algoritmo;
- a variação nunca muda;
- o sistema ficaria artificialmente complexo.

Este é o alerta de **overengineering**: aplicar o padrão sem um problema real só complica o código. A pergunta-guia continua sendo: *isso vai mudar?*

7.3 Exemplo do cotidiano: rota no Uber

Se no Uber (ou em qualquer app de mapas) for preciso **mudar a rota**, isso pode ser **Strategy**: o cálculo da rota pode seguir estratégias diferentes (mais rápida, mais curta, evitando pedágio), e a escolha pode mudar **durante** o uso. O planejador de rotas é o `Context`; cada critério seria uma `ConcreteStrategy`.

8. Strategy dentro do MVC

No MVC, o **Controller** usa **Strategy** para definir comportamento (salvar, validar, enviar dados) **sem alterar a View**. Os exemplos vistos: o **mesmo botão** de pagamento com **ações diferentes** conforme a estratégia (Pix, cartão de crédito, cartão de débito).

Também é a solução do exemplo da Aula 01 (`if (tipo == "boleto") ... else if (tipo == "cartao")`): cada forma de pagamento vira uma estratégia, e `PedidoService` depende só da abstração `Pagamento`.

9. Dúvidas

Dúvida 1: "Trocar algoritmos em tempo de execução: como assim? Por que em runtime?"

Os dois "tempos" de um programa

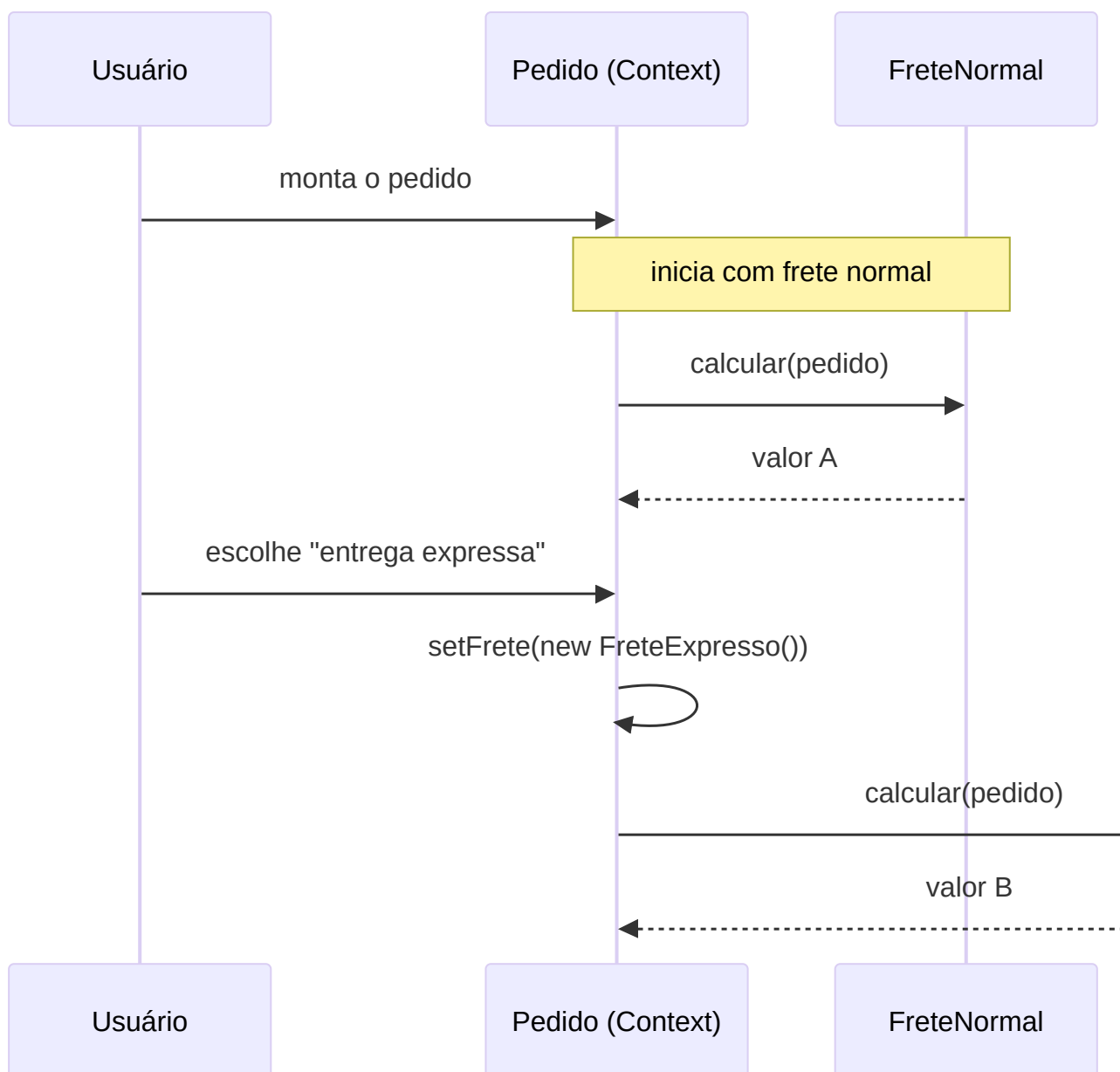
- **Tempo de compilação**: quando o código é escrito e transformado em programa. Aqui ficam definidos os **tipos**, as **classes** e as **hierarquias de herança**. O que foi decidido nesse momento **está fixo** no código.
- **Tempo de execução (runtime)**: quando o programa **está rodando**. Só agora se conhecem os **dados reais**: o que o usuário clicou, o que está no banco, qual configuração foi lida, que horas são, qual a região do cliente.

Por que herança não permite trocar

Com herança, o comportamento é definido pela **classe do objeto**, e essa escolha é feita **ao escrever o código** (`new FreteExpresso()`). Um objeto criado como `FreteNormal` **permanece** `FreteNormal` até o fim da vida dele. Para mudar o comportamento seria preciso **criar outro objeto** e copiar o estado.

Como o Strategy permite trocar

O Context guarda uma **referência do tipo da interface**. Em compilação, ele só sabe "tenho uma Strategy ", sem saber qual. Em execução, o objeto concreto **pode ser substituído** (por um `setEstrategia(...)` ou por outro parâmetro), e a próxima chamada usa o **novo algoritmo**. Quem faz isso funcionar é o **polimorfismo com ligação dinâmica**: a versão do método é escolhida pelo **objeto real** no momento da chamada.



Por que isso é útil

- **A informação só existe na execução.** O cliente escolhe "expresso" na tela, aplica um cupom, muda o endereço para outro país, ou a universidade é configurada por arquivo/banco. Não dá para saber isso ao escrever o código.
- **Sem recompilar nem reimplantar** o sistema para mudar de regra.
- **Sem if/else** espalhado decidindo qual algoritmo rodar.

Observações

- Trocar em execução é **uma possibilidade**, não uma obrigação. Se a estratégia é definida uma vez, no construtor, o padrão continua valendo pela **extensibilidade** (novas regras sem mexer no Context).
- O "por quê" mais profundo é a **separação de responsabilidades**: quem **usa** o algoritmo (Context) fica separado de quem **escolhe** e de quem **implementa** o algoritmo.

Dúvida 2: "Interface ou classe abstrata para a Strategy?"

Ver a seção 4.5. Resumo: ambas funcionam (GAMMA et al., 2000); a interface é preferida porque **não consome a única herança de classe** disponível em Java, permite **várias implementações combinadas** e não carrega estado ou código herdado (BLOCH, 2018, item 20).

Dúvida 3: "Agregação ou composição no Strategy?"

Geralmente o Context é ligado à Strategy por um **losango vazado**, ou seja, **agregação**. A razão: a estratégia **existe independentemente do** Context (pode ser criada fora, **compartilhada** por vários Context **e trocada** sem destruir ninguém), enquanto **composição** (losango preenchido) indicaria que a parte **morre com o todo**. Em resumo: no Strategy a relação é do tipo "tem um" com ciclo de vida independente.

Resumo

STRATEGY (Aula 03)

```

|
| — REVISÃO
|   | — Herança: "é um", extends, caixa-branca, polimorfismo
|   | — Composição: "tem um", caixa-preta, agregação/composição
|
| — PROBLEMA: cálculo de frete no e-commerce
|   | — if/else: muitos ifs, difícil manutenção, código gigante, alto
|   |   acoplamento
|   | — God class: mistura regra, algoritmo e controle (anti-padrão)
|   | — Herança: explosão de subclasses, rigidez, recompilação
|   | — Problema real: comportamento variável (o algoritmo muda)
|
| — SOLUÇÃO: transformar o algoritmo em objeto
|
| — PADRÃO STRATEGY (comportamental)
|   | — "Define uma família de algoritmos, encapsula cada um e os torna
|   |   intercambiáveis"
|   | — Context: usa a estratégia, só delega (atributo do tipo Strategy)
|   | — Strategy: interface comum (O QUE, não COMO)
|   | — ConcreteStrategy: implementações reais (o COMO)

```

- | |— Segredo: composição, não herança
- | |— Troca dinâmica em tempo de execução (setEstrategia)
- |
- |— EXEMPLO: controle acadêmico
- | |— CalculoNota → MediaSimples, MediaPonderada, nova regra sem mexer na Participacao
- |
- |— VANTAGENS: sem if/else, manutenção, baixo acoplamento, testes, reuso, expansão
- |— DESVANTAGENS: muitas classes, cliente conhece as estratégias, mais complexidade
- |
- |— QUANDO USAR
 - |— Sim: muitos algoritmos parecidos, comportamento varia, muitos ifs, regras novas frequentes
 - |— Não: um só algoritmo, variação nunca muda (overengineering)